

Multi-Model Recommendation Systems In e-commerce

Instructor: Md Faisal Kabir,
Members: Arjun Kumar
Brandon Beck
Blake Wilson
Vu M H Tran

COMPSC 445
Machine Learning in Applied Data Science
Spring 2026

Table of Contents

- 1. INTRODUCTION 3**
 - 1.1 Our Contribution 4**
- 2. MATERIAL AND METHODS 5**
 - 2.1 Dataset Description 6**
 - 2.2 Models Used 7**
 - 2.2.1 Content-Based recommendation..... 7
 - a. KNN with Numerical and Categorical Features 8
 - b. KNN with Text Features..... 9
 - c. KNN with Combined Features11
 - 2.2.2 User-Behavior Recommendation12
 - a. Matrix Factorization Model with Alternating Least Squares (ALS)13
 - b. LightGCN (Graph-Based Model).....15
 - 2.3 Performance Evaluation..... 17**
- 3. EXPERIMENTAL ANALYSIS:..... 18**
 - a. KNN with Numerical and Categorical Features18
 - b. KNN with Text Features.....20
 - c. KNN with Combined Features23
 - d. ALS and LightGCN:29
- 4. CONCLUSION 32**
- 5. References 35**
- 6. Appendix 36**

Abstract

In recent years, online shopping has grown rapidly, making recommendation systems an important part of modern e-commerce platforms. A well-designed recommendation system helps users discover relevant products more easily and improves their overall shopping experience. In this project, we develop several recommendation approaches to achieve two main goals: recommending products based on user behavior and recommending products based on product content. Specifically, we implement three variants of the KNN algorithm using different types of features, along with two additional models: Alternating Least Squares (ALS) and Light Graph Convolutional Networks (LightGCN). We compare the performance of these approaches in different scenarios. The results show that LightGCN and the KNN model using combined text, numerical, and categorical features achieve the best performance and effectively meet our goals. These findings provide a solid foundation for developing more advanced recommendation systems in real-world applications.

1. INTRODUCTION

E-commerce websites now offer a huge number of products; as a result, it becomes harder for users to find what they are looking for. Without proper support, users may feel overwhelmed and struggle to choose suitable products. This is where recommendation systems become important. They help users discover more relevant items based on their preferences and behavior, improving the overall shopping experience. In addition, recommendation systems are also important for businesses, as they can increase website traffic and increase sales. This can be seen in large e-commerce platforms such as Amazon and Walmart, where recommendation systems play a key role in maintaining user engagement and steady growth.

Several approaches have been widely used in recommendation systems. One common method is content-based recommendation, which uses product information such as name, description, and category to find similar items. Techniques like TF-IDF and cosine similarity are often used to measure text similarity between items, and this approach works well when user interaction data is limited [1]. However, content-based methods may struggle to capture user preferences beyond product similarity. Another popular approach is collaborative filtering, which focuses on user behavior, such as clicks, ratings, or purchases. Methods like matrix factorization, including Alternating Least

Squares (ALS), can learn hidden user-item patterns and provide personalized recommendations [2]. However, these methods often face challenges when there is not enough user interaction data, especially for new users or products. More recently, graph-based collaborative filtering models such as Light Graph Convolutional Networks (LightGCN) have been introduced. LightGCN uses the user-item interaction graph to learn user and item embeddings and improve recommendation performance [3]. While these models show strong results, they are usually more complex and require more computational resources. Although many recommendation methods have been developed, each approach still has its own limitations. Content-based methods mainly rely on product information and may not fully capture user preferences, while traditional collaborative filtering methods depend heavily on user interaction data and often struggle with new users or products. Graph-based models can improve performance, but they are more complex and not always easy to apply in practice. Because of these challenges, it is important to explore and compare different approaches to understand their strengths and weaknesses. In this project, we aim to build and evaluate multiple recommendation models using different types of data, including product content and user behavior. By comparing these approaches in the same setting, we hope to gain better insights into which methods work best and provide a foundation for developing more effective recommendation systems in real-world e-commerce applications.

1.1 Our Contribution

- We created and prepared a dataset for the recommendation task based on an e-commerce database (Vu).
- We performed exploratory data analysis (EDA) to understand better the data distribution and the main characteristics of the products and user interactions (Vu, Brandon, Blake, and Arjun).
- We applied feature engineering to transform raw product information into useful inputs for recommendation models (Vu, Brandon, Blake, and Arjun).
- For product-content recommendation, we built and tested three different KNN-based approaches:
 - a. KNN using numerical and categorical features (Blake)
 - b. KNN using text features (Brandon)
 - c. KNN using a combination of numerical, categorical, and text features (Arjun)
- For user-behavior recommendation, we implemented two collaborative recommendation models: ALS and LightGCN (Vu).

- We compared the performance of these approaches to understand their strengths and weaknesses in different recommendation scenarios (Vu, Brandon, Blake, and Arjun).
- Finally, we together visualize the output (recommendations) from each model using the e-commerce website developed by Vu.

2. MATERIAL AND METHODS

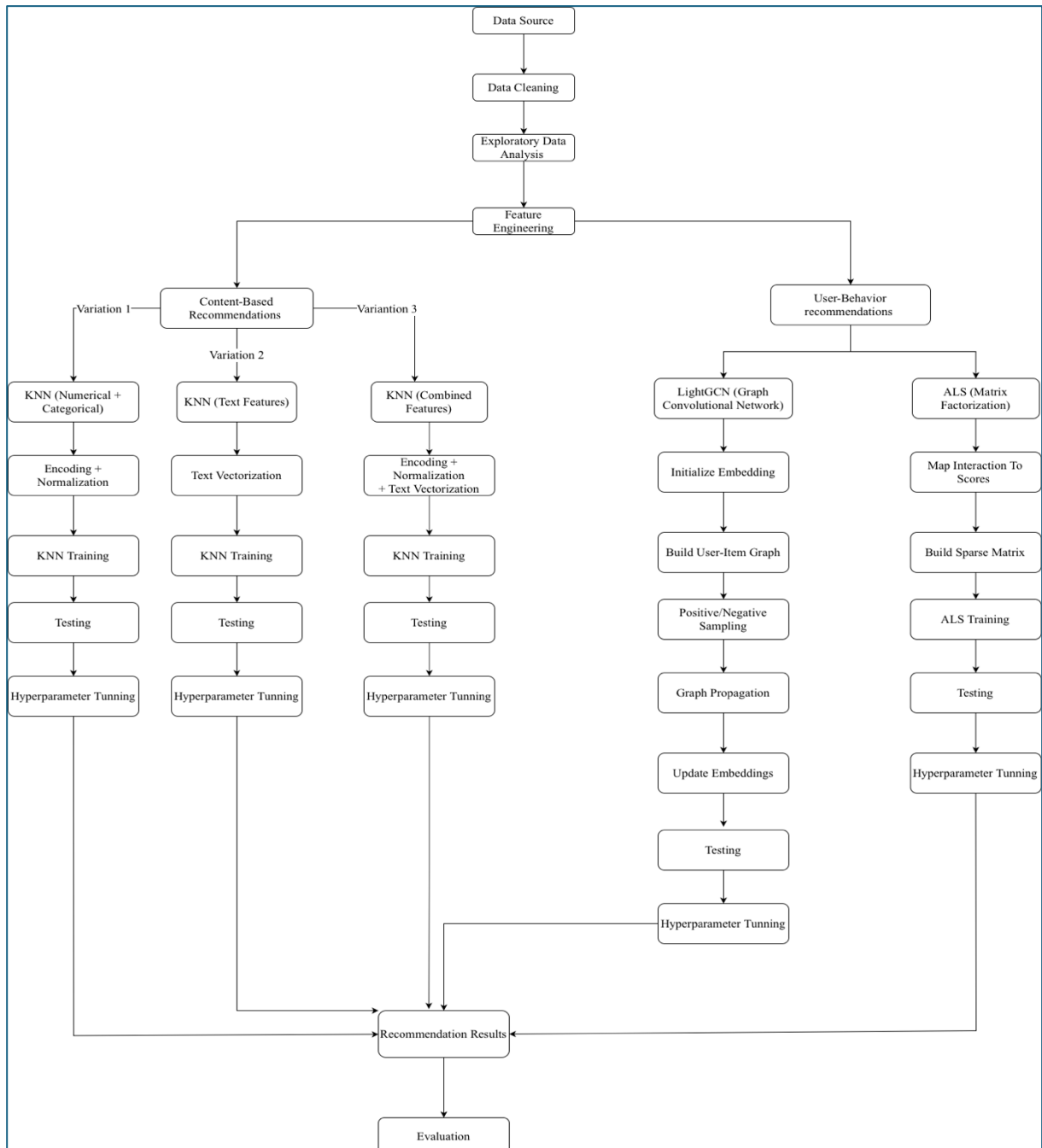


Figure 1: Multi-Model Recommendation System Pipeline

The process begins with collecting data from the source, followed by data cleaning and exploratory data analysis (EDA) to better understand the dataset. After that, feature engineering is applied to prepare the data for different recommendation approaches.

The system is then divided into two main parts. The first part focuses on content-based recommendation, where three variants of the KNN algorithm are developed using different types of features, including numerical and categorical data, text data, and a combination of all features. Each approach involves feature transformation, model training, testing, and tuning.

The second part focuses on user-behavior-based recommendation. In this part, two models are implemented. The first model is ALS, which uses user interaction data to build a sparse matrix and learn latent representations. The second model is LightGCN, which represents user-item interactions as a graph, initializes user and item embeddings, and updates them via graph-based propagation. Both models are trained, tested, and tuned to improve performance.

Finally, the outputs from all models are used to generate recommendation results, which are then evaluated and compared.

2.1 Dataset Description

The dataset used in this project was collected from an e-commerce database, a personal project. It includes two main parts: product information and user interaction data. This dataset was chosen because it reflects a real online shopping environment, where both product details and user behavior are important for building recommendation systems.

The product dataset contains 244 records, each with distinct attributes describing the product. These include basic information such as name, brand, and category, as well as price and product descriptions. Text fields like short and long descriptions help capture the meaning of the product, while numerical and categorical features such as price and category help describe its structure.

The main product features are:

- `product_id`: unique identifier for each product
- `product_name`: name of the product
- `brand_name`: brand of the product
- `category_name`: main category
- `sub_category_name`: sub-category
- `price`: original price
- `product_type`: type of product (such as featured or new)
- `short_description`: short text description
- `long_description`: detailed description

The user interaction dataset contains 10,000 records, where each record represents an interaction between a user and a product. These interactions reflect user behavior and preferences, which are important for collaborative recommendation models.

The main interaction features are:

- `user_id`: unique identifier for each user
- `product_id`: identifier of the product
- `interaction_type`: type of interaction (click, add to cart, add to Wishlist, and rating).

In this project, different types of data are used depending on the model. For product content-based recommendation, product features in the product dataset, such as text, category, and price, are used to find similar items. For user-behavior-based (collaborative) models, interaction features in user interaction dataset are used to learn user preferences and recommend relevant products. For more details about the dataset we used, please refer to the project GitHub repository, [Product dataset](#), and [user-item interaction dataset](#).

2.2 Models Used

2.2.1 Content-Based recommendation

We use the K-Nearest Neighbors (KNN) algorithm to recommend products based on similarity between items. The main idea is to find products that are closest or most similar to a selected product.

In this project, we implement three different KNN approaches. Each approach uses different feature engineering steps and different preprocessing techniques. For example, one approach may focus more on product categories and price, while another approach may focus on text information such as product name and

description (details about approaches and techniques will be mentioned below). Although the preprocessing steps are different, all approaches eventually produce the same final structure, called a feature matrix. This matrix contains all products as rows and the processed features as columns.

After creating the feature matrix, we use the KNN model from Scikit-learn to fit the data. In this case, KNN is used as an unsupervised learning model, not a supervised classification model. This means we are not training the model to predict a label. Instead, the model learns the distance or similarity relationship between products based on the feature matrix.

After the KNN model is fitted, it can be used to generate recommendations. The process is simple: we pass the selected product into the model's ``neighbors()`` function and specify the number of nearest products we want to retrieve. The KNN model then finds the K products with the shortest distance to the selected product. The function returns the indices of those nearest products, and these indices are mapped back to the real product records in the database. These products are then shown as the recommendation results.

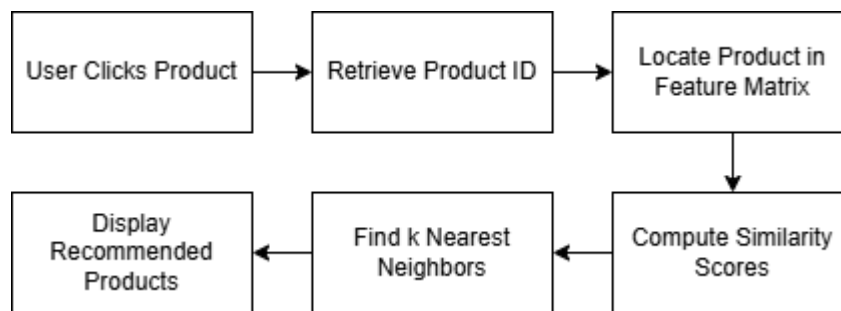


Figure 2: Recommendation Process Triggered by User Product Click

a. KNN with Numerical and Categorical Features

This KNN approach recommends products by using categorical and numerical features from the dataset to recommend similar products to users. The model uses category, sub-category, brand, and price as the input features. It focuses on product classifications and price information to measure similarity between items.

Libraries Used

This model was implemented using scikit-learn, scipy, and pandas

Diagram

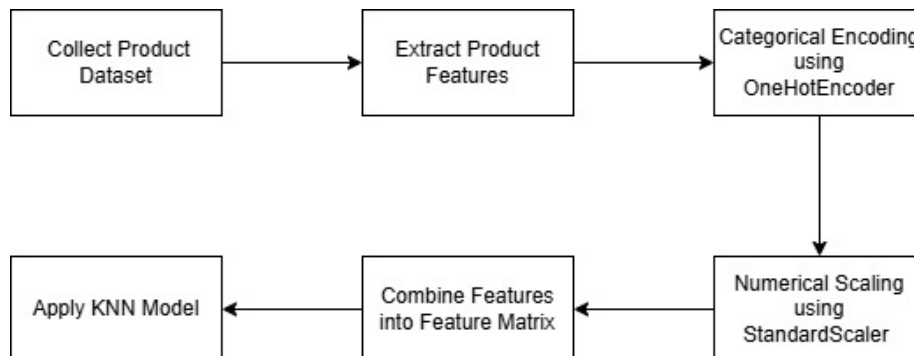


Figure 3: KNN model with Numerical and Categorical Features Flowchart

Methods

Categorical features like Category, sub-category, and brand are encrypted using One-Hot Encoding to convert each category into a binary representation, and numerical feature like price is standardized using StandardScaler, so it contributes fairly to the model. Next, both feature types are combined into a single feature matrix using horizontal stacking.

A KNN model is then trained using Euclidean distance as the distance metric. When given a product, the model identifies other similar products based on category, sub-category, brand, and price.

Key Idea

The goal is to recommend similar products to users using only categorical and numerical columns. This allows the model to recommend items that are of the same category and around the same price, without using text descriptions. For more details about the implementation of this model, please refer to the project GitHub repository, [KNN with Numerical and Categorical Features](#).

b. KNN with Text Features

This model uses only text product features like product name, brand name, and short description, and long description to train KNN. When products have similar wording in their descriptions, or have a similar brand name or product name, this model will determine those to be more similar and recommend them first.

Library Used

This model was implemented using scikit-learn and pandas libraries.

Diagram

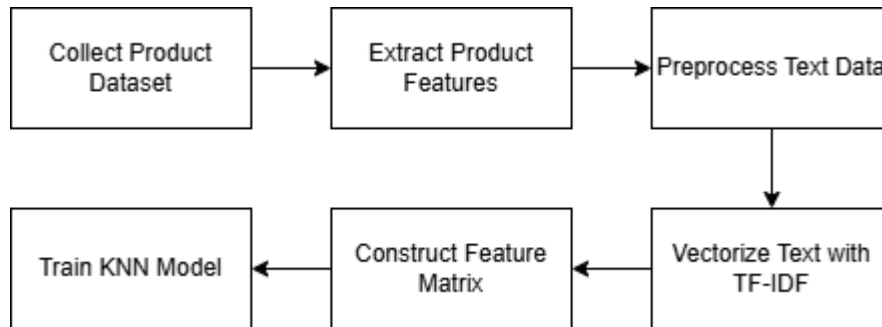


Figure 4: KNN model with Text Features Flowchart

Method

The process starts by taking the data from all relevant product features, converting and concatenating the values into one string, with empty text replacing missing values. Repeated weighting was applied to product and brand names to give them importance over the short and long descriptions.

Next, a feature matrix was created using the `TfidfVectorizer` from `scikit-learn` where rows represent products and columns represent features. Unigrams and bigrams were used to represent single words and small phrases as features. Stop words were removed to eliminate words that appear too frequently and are not informative, reducing noise in the data. A predefined set of stop words (`ENGLISH_STOP_WORDS`) from `scikit-learn` was used to remove common words such as 'the', 'and', and 'is'. In addition, a custom set of terms was included to further improve the quality of the recommendations.

Examples of these words include 'product', 'item', 'great', 'good', 'quality', and 'best'. Additionally, minimum and maximum document frequency thresholds (`min_df/max_df`) were applied to automatically eliminate words that appeared too frequently or infrequently.

Finally, a KNN model was trained using cosine similarity as the distance metric. Using cosine similarity allows the model to find the k nearest neighbors based on similarity between feature vectors.

Parameters/Hyperparameters Used

The number of k neighbors was set to a maximum of eleven and adjusted based on the size of the dataset to ensure effective similarity comparisons.

Key Idea

The key idea for this model is to convert text into a numerical value so the model can analyze it and recommend other similar products based on the results. For more details about the implementation of this model, please refer to the project GitHub repository, [KNN with Text Features](#).

c. KNN with Combined Features

This approach of KNN combines multiple types of product features, including text, categorical, and numerical data, to improve the recommendation performance. Instead of relying on a single feature type, this method integrates all relevant product information to ensure a general representation of each item. From the two models above, we notice that using only one category type, like text or category, may limit the model's ability to fully understand the product similarity. By combining text descriptions, categorical attributes, and numerical values, the model can generate a more accurate and meaningful recommendation. directly.

Library Used

This model was implemented using scikit-learn, SciPy, and pandas library.
Diagram

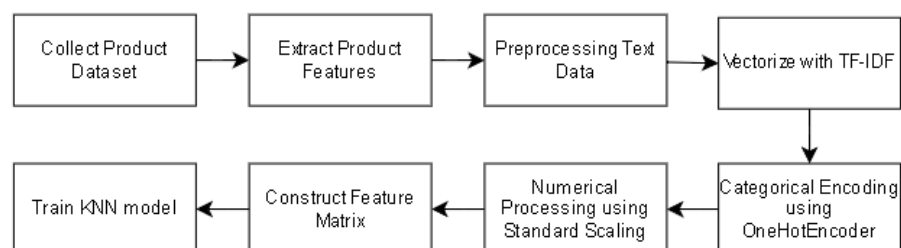


Figure 5: KNN model with Combined Features Flowchart

Method

This process starts with data preprocessing and feature engineering: Text Features include product names, short descriptions, and long descriptions. TD-IDF vectorization is then applied to convert the text into numerical representations. Additional weighting is applied to product names to increase their importance. Categorical Features such as brand name, category, and sub-category are encoded using One-Hot Encoding to transform them into a numerical format. Numerical Feature like price is scaled using the Standard

Scaler to normalize their values. The scaled data is then converted into a sparse matrix format.

Next, all feature types are combined into a single feature matrix using horizontal stacking. This combined matrix represents each product in a high-dimensional space.

A KNN model is then trained using cosine similarity as the distance metric. For a given product in the dataset, the model identifies the most similar products based on their combined features.

Key Idea

The goal is to compute similarity between the products using descriptions, category, brand, and price. This allows the model to recommend items that are similar across multiple dimensions rather than just one. For more details about the implementation of this model, please refer to the project GitHub repository, [KNN with Combined Features](#).

2.2.2 User-Behavior Recommendation

What is User-Behavior-Based Recommendation?

User behavior-based recommendation, also known as collaborative filtering, is an approach where the model predicts what you will like by seeing what other people with similar tastes to you.

What is a Matrix Factorization model?

A model that breaks/factorizes the user-item interaction matrix into 2 smaller matrices, the user matrix (user factors) and the item matrix (Item¹ factors), and then uses them to predict what a user may like

Popular methods used in the model: Stochastic Gradient Descent, Singular Value Decomposition, Alternating Least Squares, ...

What is a Graph Neural Network model?

A model that receives a user-item interaction dataset as a graph, where user and item are nodes and interactions are edges, “forcing” nodes to learn mutually and then use all the information that was learned to predict what a node represents for user may like.

How do we use both models to obtain recommendations?

Although these two models learn in different ways, they both return the embeddings of all users and all products after training.

¹ The words “item” and “product” are interchangeable in this report.

In our project, when a user clicks on any item, we use the embedding of that clicked item as a temporary user embedding. This temporary embedding represents the user's current interest at that moment. Then, we compare it with the embeddings of the remaining products.

To calculate the recommendation score, we use the dot product between the temporary user embedding and each product embedding. A higher dot product means the product is more related to the user's current interest. After that, we sort the products by score and select the top K products as the recommendation results.

a. Matrix Factorization Model with Alternating Least Squares (ALS)

ALS (Alternating Least Squares) is a collaborative filtering method that learns user preferences from interaction data. Instead of using product features, this model focuses on how users interact with products and uses a matrix factorization approach.

Why we use ALS model²

ALS can learn hidden patterns from user behavior. In our project, users interact with products through actions such as clicking, adding to the Wishlist, adding to the cart, and rating. These actions show different levels of interest, and ALS can use them to understand what products a user may like.

Compared with the Stochastic Gradient Descent (SGD) method, ALS is usually more stable because it does not depend as much on small step-by-step updates. SGD can be powerful, but it often needs more careful tuning, such as learning rate and number of training epochs. Also, ALS is better suited for sparse datasets, whereas the Singular Value Decomposition (SVD) method requires a dense matrix. In the real world, not every user gives a clear rating; as a result, the user-item interaction dataset is usually very sparse. A user may only interact with a few products, while the website may contain hundreds or thousands of products, and ALS works well in this situation

Library Used

This model was implemented using the Implicit library along with Pandas and NumPy.

² To be precise, ALS is just a method, the true model is Matrix Factorization. But in this report, we will consider ALS as a model.

Diagram

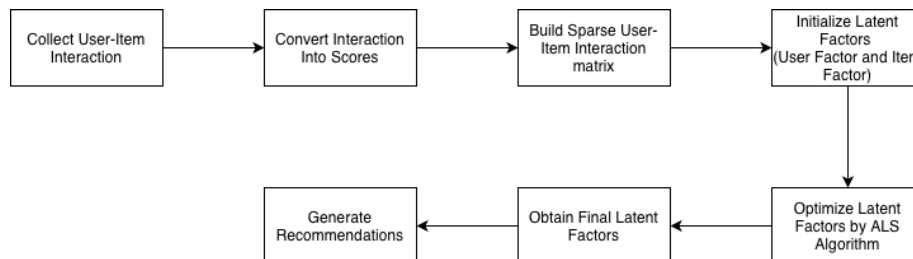


Figure 6: Matrix Factorization Model Flowchart

Method

The process starts by converting user interactions into numerical scores. Since the dataset is based on implicit feedback, there are no explicit ratings from users. Instead, different types of interactions are assigned different weights to represent their importance. In this project, the interaction weights are defined as:

- click: 0.05
- add to Wishlist: 0.7
- add to Cart: 1.0
- rating (R1-R5): Rating level

Next, a user-item matrix is built, where each row represents a user, each column represents a product, and each value represents the interaction score. This matrix is usually sparse because most users only interact with a small number of products.

ALS then factorizes this matrix into two smaller matrices: a user embedding matrix (also known as user factors) and an item embedding matrix (also known as item/product factors). The model works by repeatedly updating one matrix while keeping the other fixed. In other words, it first updates user embeddings based on item embeddings, then updates item embeddings based on user embeddings. The updating is using the Normal Equation³. This alternating process continues until the model becomes stable.

Key Idea

The goal is to approximate:

³Normal Equations or Ordinary Least Squares are a closed-form method for solving linear least squares problems, minimizing the sum of squared differences between observed and predicted values.

$$R \approx U \times V^T$$

Fix item embeddings (item factors) and update/solve user embeddings (user factors):

$$u_u = (V_u^T V_u + \lambda I)^{-1} V_u^T R_u$$

Figure 7: Normal Equation for Solving User Factors

Fix user embeddings (user factors) and update/solve item embeddings (item factors):

$$v_i = (U_i^T U_i + \lambda I)^{-1} U_i^T R_i$$

Figure 8: Normal Equation for Solving Item Factors

where:

- R: user-item interaction matrix,
- U: user embeddings,
- V: item embeddings.
- λ : a regularization parameter to prevent overfitting.
- I : the identity matrix.

The hyperparameters used:

- factors: 50 — number of latent features used to represent users and items
- regularization: 0.05 — controls overfitting by penalizing large values
- iterations: 100 — number of times the model updates user and item matrices
- alpha: 10 — controls the confidence level of interaction strength in implicit feedback

For more details about the implementation of this model, please refer to the project GitHub repository, [Matrix Factorization With ALS](#).

b. LightGCN (Graph-Based Model)

LightGCN is a graph-based recommendation model that learns from the relationships between users and products. Instead of using a matrix factorization approach, it represents the recommendation problem as a graph.

Why we use LightGCN

LightGCN model is a simpler and lighter version of a traditional Graph Convolutional Network (GCN). A normal GCN usually uses extra operations such as feature transformation and activation functions. LightGCN removes these parts and keeps only the most important idea: passing information between connected users and products. Because of this, LightGCN is easier to train and more focused on recommendation tasks.

LightGCN is also useful when interaction data is limited, users only interact with items by click, the graph structure can still provide useful information from nearby users and products. As a result, LightGCN can produce personalized recommendations by learning from both direct and indirect connections in the interaction graph

Libraries Used

This model was implemented using PyTorch and PyTorch Geometric (PyG).

Diagram

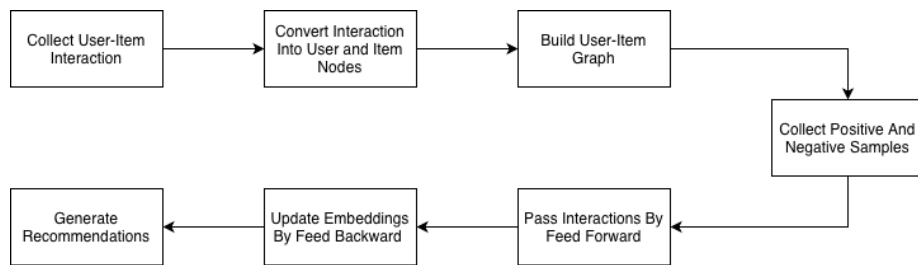


Figure 9: LightGCN Model Flowchart

Method

First, user-product interactions are used to build a graph. In this graph, users and products are nodes, and interaction is a click. Next, each user and product is given a random embedding at the beginning. During the feed-forward process, LightGCN passes information through the graph. A user embedding receives information from the products the user interacted with, and a product embedding receives information from users who interacted with it. Through multiple layers, the model can learn both direct and indirect relationships.

During training, the model uses positive and negative sampling. Positive items are products the user has interacted with, while negative items are products the

user has not interacted with. The model learns to give higher scores to positive items than negative items.

After the feed-forward step, the model calculates a loss value to measure the error. During the feed backward process, the model uses this error to update the user and product embeddings. This process is repeated many times until the loss converges. Finally, the trained embeddings are used to rank products and return the top recommendations.

Key Idea

The model is trained using Bayesian Personalized Ranking (BPR) loss:

$$\sum_{(u,i,j) \in D_s} \ln \sigma(\widehat{r}_{ui} - \widehat{r}_{uj}) - \lambda(\|U\|^2 + \|I\|^2)$$

where:

σ : the logistic sigmoid function.

D_s : the set of triplets (u,i,j) , such that user u has interacted with item i (positive item) but not with item j (negative item).

λ : a regularization parameter to prevent overfitting.

$\|U\|^2$ and $\|I\|^2$: the regularization terms for the user and item matrices.

$\widehat{r}_{ui}, \widehat{r}_{uj}$: Predicted rating from user u to interacted item i and from user u to non-interacted item j .

The intuition from BPR is to make the model give a higher score to the positive item than the negative item.

The hyperparameters used:

- embedding_dim: 64 — size of the vector used to represent each user and item
- num_layers: 3 — number of graph propagation steps
- batch_size: 30 — number of samples used in each training step
- epochs: 500 — number of full training iterations
- learning_rate: 0.01 — step size used to update model parameters
- Optimization: Adam – Faster convergence when updating parameters

For more details about the implementation of this model, please refer to the project GitHub repository, [LightGCN](#).

2.3 Performance Evaluation

In this project, the model performance is mainly evaluated through website visualization. After the recommendation models return the top K recommended products, the results are displayed directly on the e-commerce website. This

allows us to observe whether the recommended products match the user's recent interactions and interests.

To evaluate the user-behavior recommendation models, we tested different user behavior scenarios. For example, we created a new user and interacted with only one product to see how the models perform with very sparse data. Then, we interacted with more products from the same category or brand to see whether the recommendations became more accurate and focused.

We also evaluated the product content-based recommendation models through the website. For this part, we clicked on any product, scrolled down to the recommendation section, and observed the products returned by the three KNN variation models. This helped us compare whether each model could recommend products that were similar to the selected item based on product information such as category, brand, price, and description.

The recommended products from different models are shown on the website, making it easier to compare the results visually. If the recommended products are closely related to the selected product or the user's interaction history, the model is considered to perform well.

3. EXPERIMENTAL ANALYSIS:

a. KNN with Numerical and Categorical Features

The KNN model using numerical and categorical features was used in our project to measure how well product recommendations can be made without using text-based data. This model focuses on categorical and numerical features, including category, sub-category, brand, and price.

Results Overview

The model was able to group products that share the same category, sub-category, and brand, especially when they fell in similar price ranges. In many cases, the recommendations given were related to the initial product and looked like they could be actual recommendations given by a more advanced model.

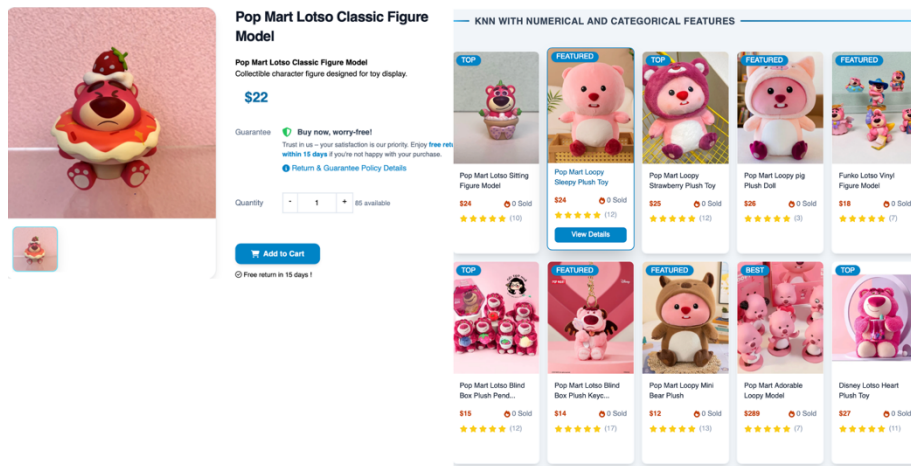


Figure 10: Example of Recommendations for Pop Mart Lotso

All the recommendations (Figure 9) given by the model look realistic for this item. However, a key issue that we observed during testing was that the model would sometimes recommend the exact same product as the user initially selected. This occurred when multiple products shared the exact same category, sub-category, and brand, and had similar or exact prices. As a result, the feature vectors for some of the products were identical, so the model had a difficult time distinguishing between the products and would return the same exact product.

Model Weakness

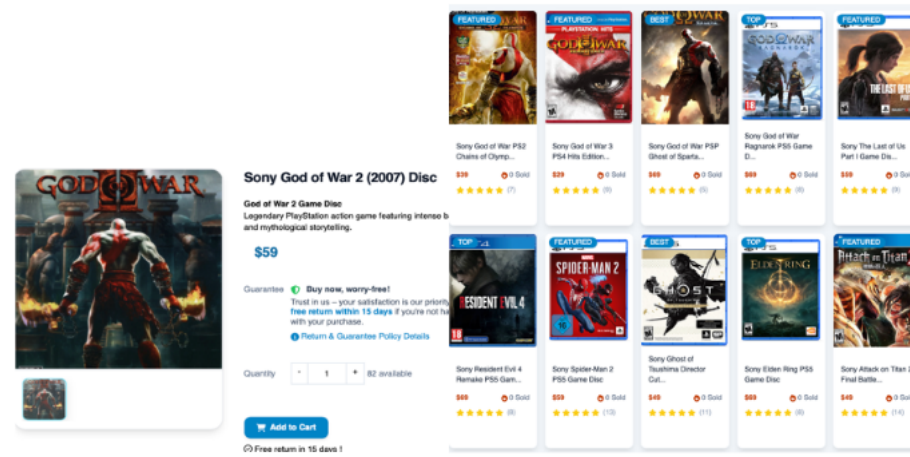


Figure 11: Example of Recommendations for GOW 2

In the recommendations list for “Sony God of War 2 (2007) Disc” (GOW 2) item, we can see that this item is repeated. This is an error that occurs due to all the products sharing the same category (Electronics), sub-category (Game Disc), and brand (Sony). Also, as shown in Figure 5, the model lists all the \$59 Sony games first, then the \$69 games.

b. KNN with Text Features

Results Overview

The performance of the model was evaluated by analyzing the similarity scores and the relevance of recommended products. We tested the model multiple times using different products and observing the recommendations. The products with similar names, brand names, and keywords in the descriptions were consistently recommended and given higher similarity scores.

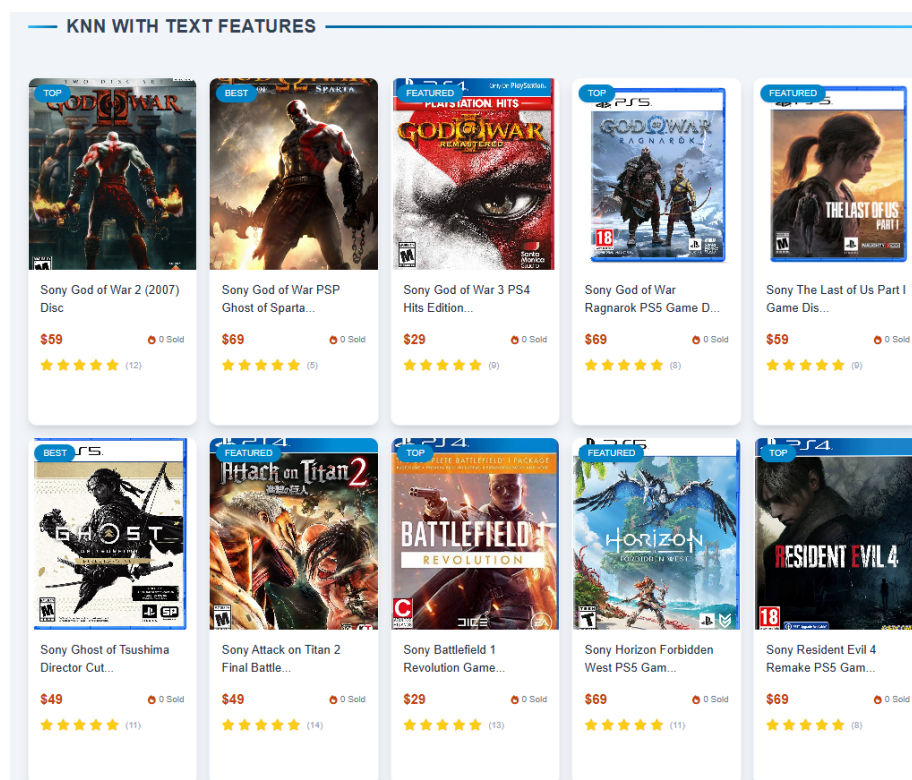


Figure 12: Recommended Products like GOW 2

The recommendations above show the results of searching for the product God of War PS2 Chains of Olympus (GOW 2). This highlights the strength of this model because there are several other products that share many of the same words or phrases. The highest-ranked recommendations are other God of War titles, which share identical or highly similar keywords in their names. As the similarity score decreases, the model begins to recommend other Sony video games that still share relevant terms such as platform or genre, demonstrating that the model maintains meaningful similarity even beyond exact title matches. This behavior is a result of the TF-IDF representation, where shared keywords such as 'God of War' are given high importance, leading to strong similarity between related products.

Model Weakness

The KNN text-based model performs well when the product descriptions contain important and meaningful information, as it can better identify similarities based on keywords and phrases. However, since it only takes text into consideration, it may fall short when comparing certain products that have

similar wording but are completely different. It is possible to address some of the issues with the text-based model by removing more stop words to further reduce noise; however, some words that appear frequently can also be meaningful. By removing these words, it could inhibit the model's ability to capture important differences between products.

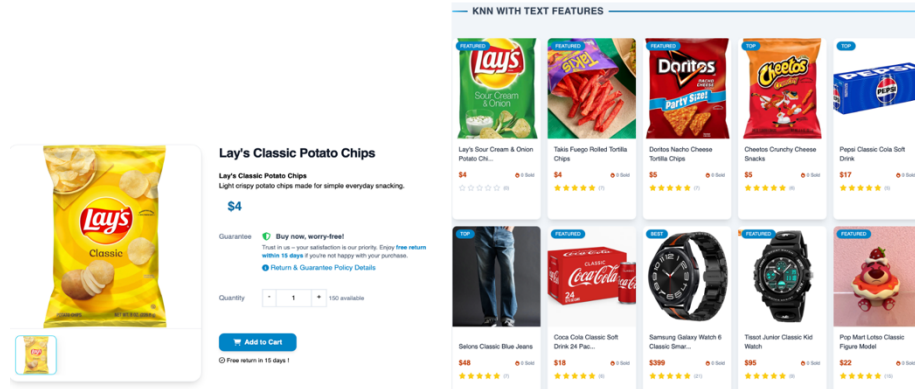


Figure 13: Recommend Products like Lay's Classic

From the outcomes (Figure 12) above, the model generated recommendations for the product Lay's Classic Potato Chips. The first four recommendations are fine, but as more suggestions returned, the word "classic" confuses the model. Because the model assigns importance to frequent terms like "classic" within a product's name or description, it will group products based on shared words instead of how similar the products actually are.

c. KNN with Combined Features

Results Overview

The performance of the model was evaluated by looking at similarity scores and how relevant the recommended products were. We tested the model using different products and observed the results. The model consistently recommended products with similar names, descriptions, categories, and brand names, which also had higher similarity scores. Including features like category and price helped improve the recommendations, since products in the same category and a similar price range were more likely to be suggested. Overall, combining different types of features helped the model produce more accurate and useful recommendations.

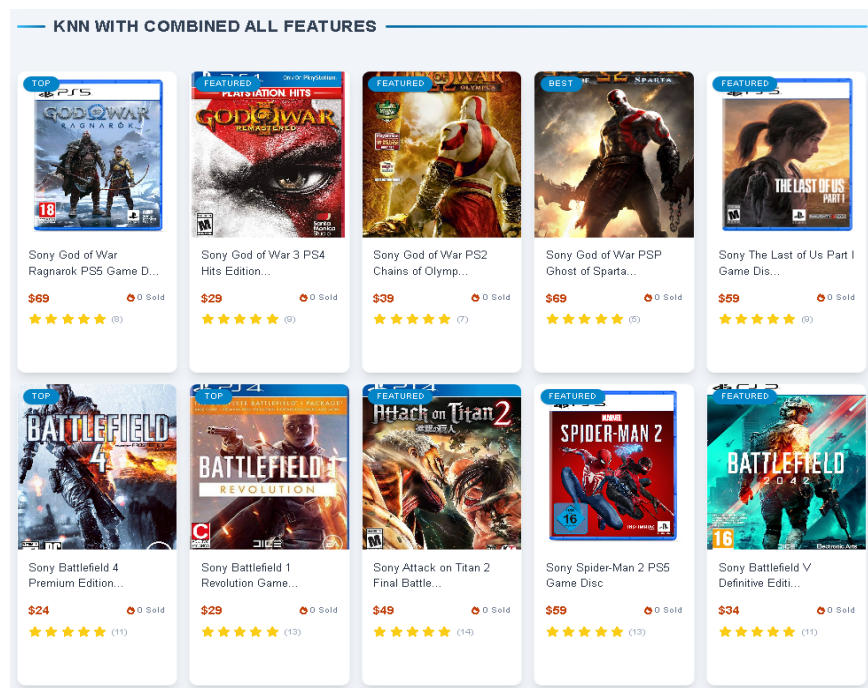


Figure 14: Example Recommendations for “God of War PS2 Chains of Olympus”

The KNN model with combined features performs well because it uses multiple types of information, including text, category, and price, to find similar products. This allows the model to generate more accurate and well-rounded recommendations compared to using only text-based features. Like the text-based model, it does not rely on user data, so it works well even when user

interaction data is not available. However, the model may still have limitations if some features are missing or not well represented. Overall, combining multiple features helps reduce errors and improves recommendation quality.

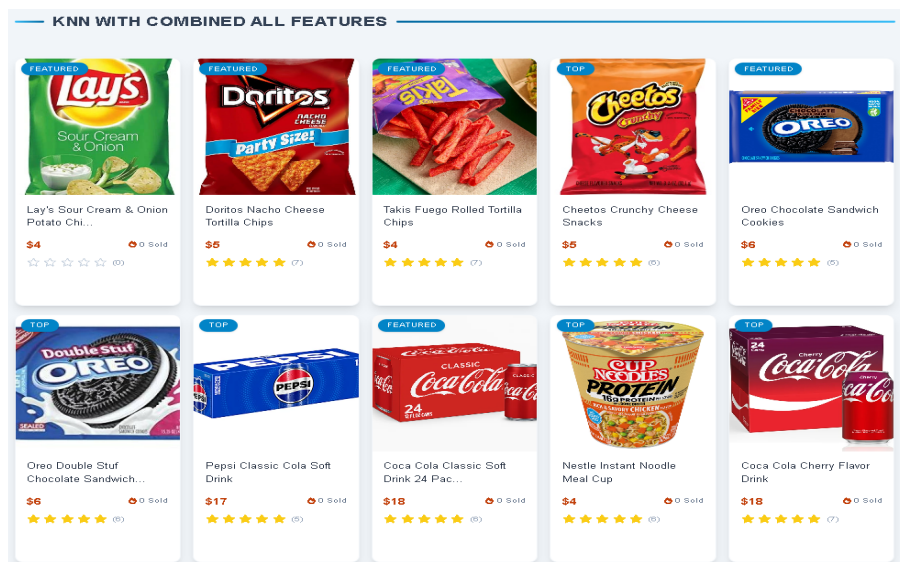


Figure 15: Example of Recommendations for “Lay’s Classic Potato Chips”

In figure six, the model generated recommendations for the product Lay’s Classic Potato Chips. The recommendations show that the model correctly groups similar snack items such as Doritos, Cheetos, and Takis, which share similar categories and product types. Higher similarity scores are given to products that are closely related in both description and category. As the similarity score decreases, the model begins to recommend items that are still somewhat related, such as drinks or other packaged foods. This behavior shows that the model is not only using text, but also category and price information to make more balanced recommendations. Overall, the model maintains meaningful similarity across different types of products by considering multiple features instead of relying on just one.

Convergence Analysis

The KNN combined features model does not use an iterative training process, so convergence analysis is not applicable. The model is built by transforming the data into a combined feature matrix and fitting the KNN algorithm once. Because of this, the results remain stable after the model is created.

Model Comparison

From the experiments, we observe that each KNN model performs differently depending on the types of features used. The KNN model with text features performs well when product descriptions contain meaningful keywords. It is effective at identifying products with similar wording, but it may sometimes recommend items that share similar text but are not actually similar in functionality or category.

The KNN model with numerical and categorical features provides more structured recommendations based on product attributes such as category and price. This model works well when products are clearly grouped, but it may miss important similarities if descriptive text is not considered.

The KNN model with combined features performs the best overall because it uses text, categorical, and numerical data together. This allows the model to have both the meaning of the product and its structure, leading to more accurate and well-balanced recommendations. It reduces the weaknesses of the individual model by considering multiple aspects of each product.

This shows that combining different types of features improves recommendation quality. While single-feature models can perform well in specific cases, the combined model gives more consistent and reliable results with different types of products.

Case 1: KNN Model with Text Features

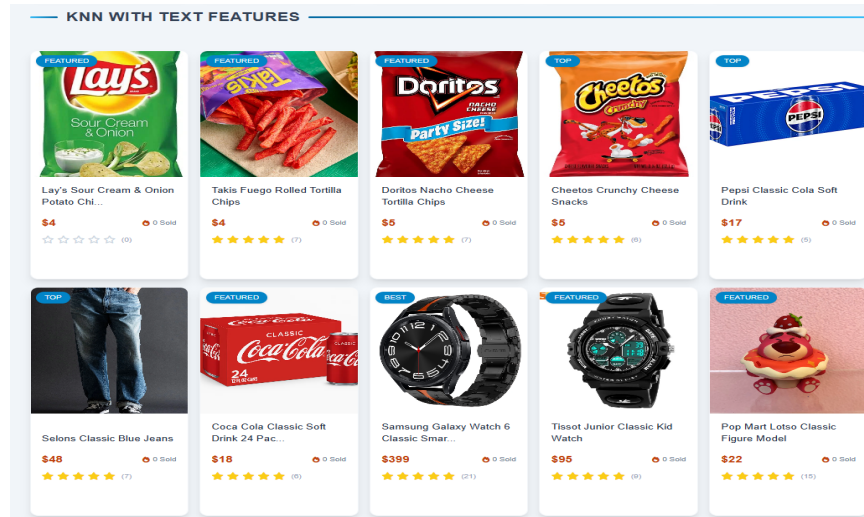


Figure 16: Example showing the weakness of Recommendations for “Lay’s Classic Potato Chips”

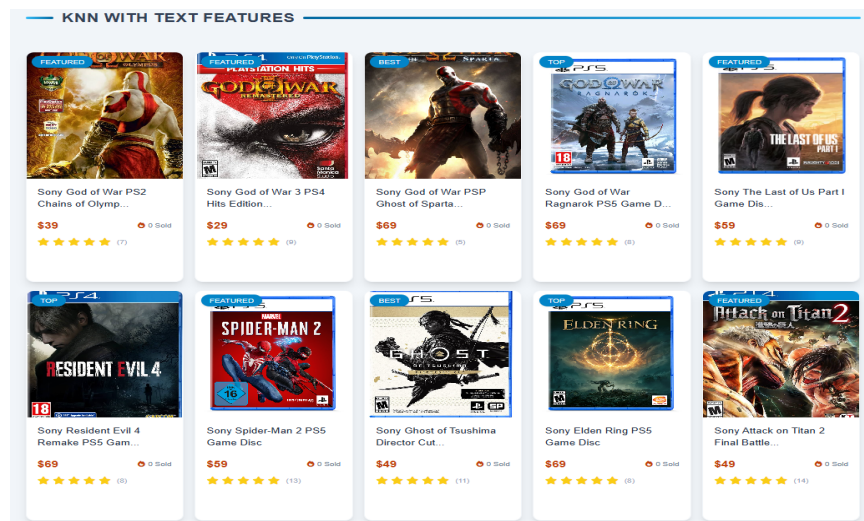


Figure 17: Example showing the strength of Recommendations for “God of War PS2 Chains of Olympus”

In this case, the text-based model correctly recommends similar products when keyboard matches closely. However, it may also recommend items that share similar wording but are not truly related, showing a limitation of relying on text.

Case 2: KNN Model with Numerical and Categorical Features

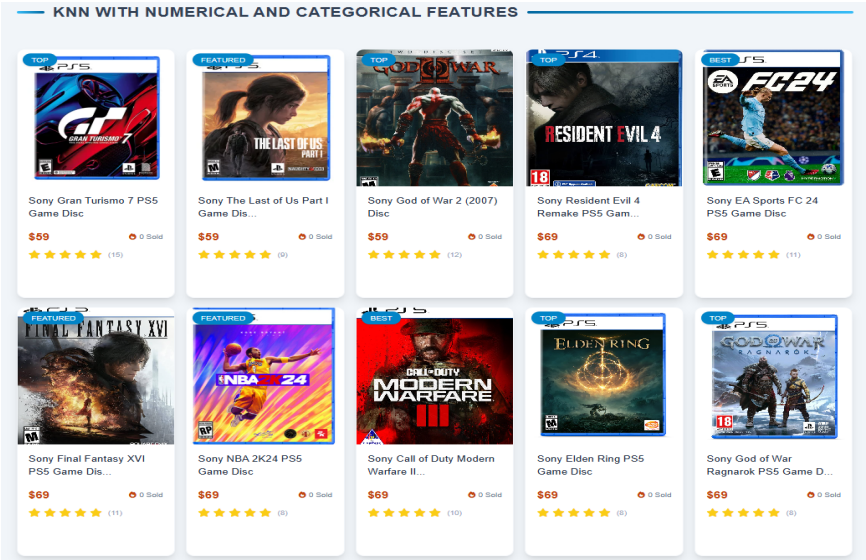


Figure 18: Example showing the weakness of Recommendation for “God of War PS2 Chains of Olympus”

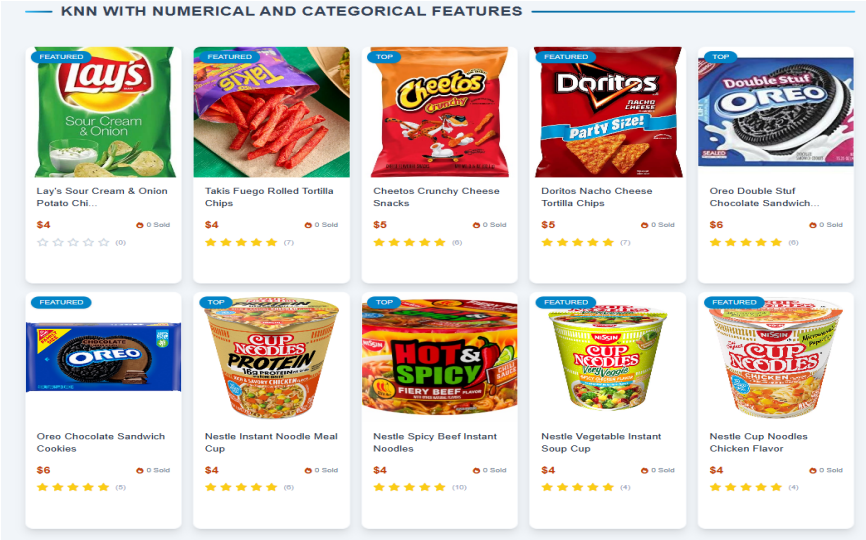


Figure 19: Example showing the strength of Recommendations for “Lay’s Classic Potato Chips”

In this case, the model performs well when products belong to the same category and price range. However, it may miss relevant items that are similar in meaning but differ slightly in category or structure, since it does not use text information.

Case 3: KNN Model with Combined Features

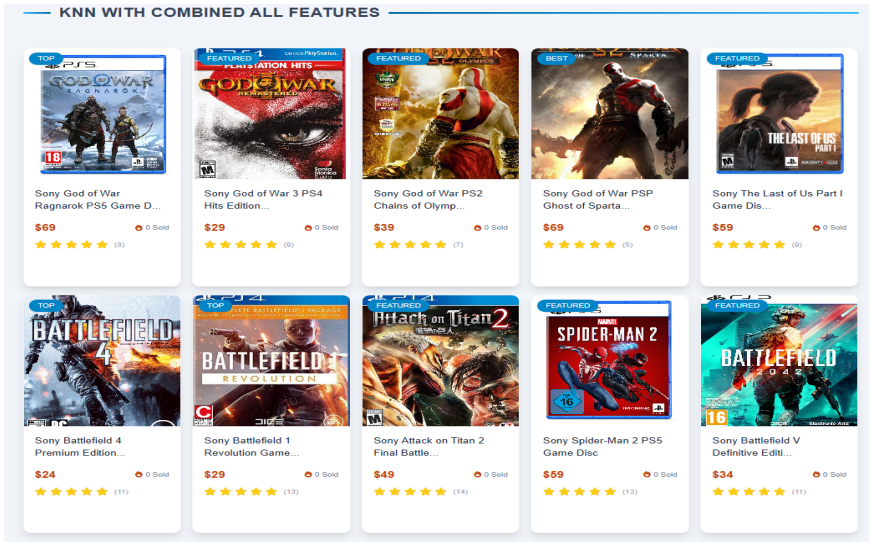


Figure 20: Example showing the strength of Recommendations for “God of War PS2 Chains of Olympus”

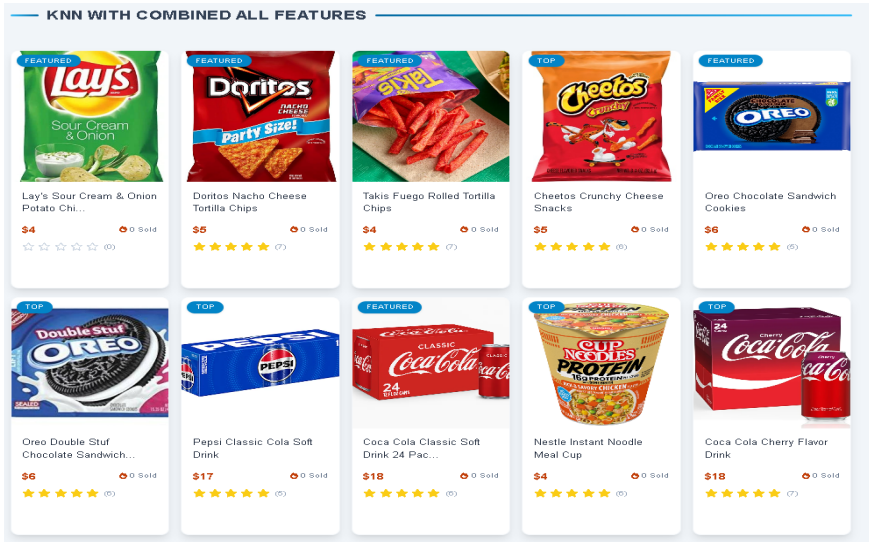


Figure 21: Example showing the strength of Recommendations for “Lay’s Classic Potato Chips”

In this case, the combined model provides accurate and well-balanced recommendations by considering text, category and price together. The model consistently suggests relevant products and avoids many of the errors seen in other models.

These cases further support that combining multiple feature types lead to more consistent and accurate recommendations.

d. ALS and LightGCN:

Results Overview

The performance of the models is evaluated using visualization from the [e-commerce website](#). Multiple experiments were conducted by changing model settings, such as hyperparameters, and observing their impact on performance.

Hyperparameter Analysis

We also tested different hyperparameter values to see how they affect performance. For example, in LightGCN, we varied the number of layers and embedding size. We observed that increasing the number of layers helps the model capture more information from the graph, but too many layers may lead to over-smoothing. Similarly, for ALS, increasing the number of factors improves the model's ability to learn user preferences, but may also increase training time.

Model Comparison

From the experiments, we observe that LightGCN performs better than ALS when user interaction data is sparse, such as in the case of new users with very few interactions. This is because LightGCN uses the graph structure to share information between users and items, allowing it to learn even with limited data.

On the other hand, ALS performs very well when the interaction data becomes dense, meaning users have interacted with many products over time. In this case, ALS can learn strong user preferences and provide accurate recommendations.

This shows that each model has its own advantage depending on the data condition. LightGCN is more suitable for cold-start or sparse scenarios, while ALS is effective in long-term usage where sufficient interaction data is available.

Case 1: New User Interacted With iPhone 17 Only

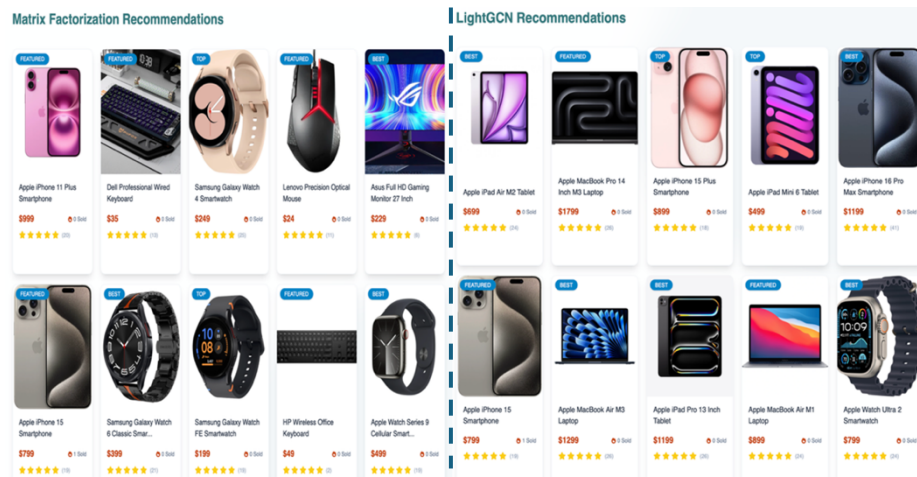


Figure 22 : New User Interacted With iPhone 17 only

On the left, Matrix Factorization recommends some related Apple products, but it also includes less related items, such as a keyboard, mouse, monitor, and Samsung watches. This shows that with only one interaction, Matrix Factorization may not understand the user's preference clearly yet.

On the right, LightGCN gives more focused recommendations, mostly Apple-related products such as iPhones, iPads, MacBooks, and Apple Watch. This shows that LightGCN works better in a case where there are too few interactions from a new user.

Case 2: New User Interacted With iPhone 17, iPhone 16, iPhone 15, iPad Mini.

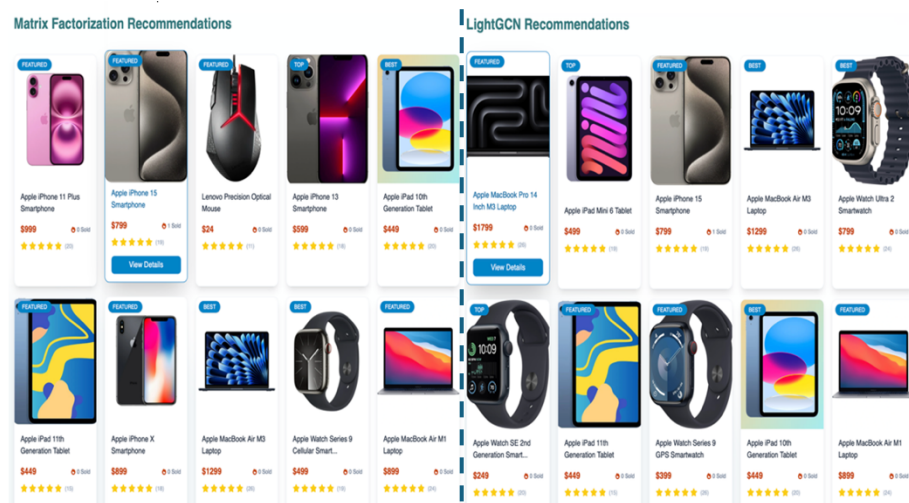


Figure 23 : New User Interacted with More Apple Products

On the left, Matrix Factorization improves compared with the previous result. Since the user now has more Apple-related interactions, the model can understand the user's preference more clearly. As a result, it recommends more Apple products, such as iPhones, iPads, MacBooks, and Apple Watches. However, it still includes one less related item, such as the mouse, which shows that the model is not fully focused yet.

On the right, LightGCN also recommends mostly Apple-related products. Overall, both models perform better when more interaction data is available, but LightGCN still gives slightly more focused recommendations in this example.

Convergence Analysis

During training, we also observed how the models converge over time. For LightGCN, the training loss decreases steadily as the number of epochs increases, showing that the model is learning effectively. After a certain number of epochs, the improvement becomes smaller, indicating that the model has reached a stable state.

4. CONCLUSION

This project developed a recommendation system for an e-commerce website by combining both product content-based recommendation and user behavior-based recommendation. The main goal was to improve the shopping experience by helping users discover products that are more relevant to their interests.

For the product content-based recommendation part, we used different KNN variation models. These models recommend similar products based on product information such as category, brand, price, product name, and product description. By clicking on a product and observing the recommendation section, we can see whether the system returns products that are similar to the selected item.

For the user behavior-based recommendation part, we implemented Matrix Factorization using the Alternating Least Squares method and LightGCN. These models use user-item interaction data, such as clicks, add to Wishlist, add to cart, and ratings, to learn user preferences. ALS learns hidden patterns from the interaction matrix, while LightGCN learns from the graph structure between users and products.

These models were connected to the e-commerce website so that the recommendation results could be displayed and tested visually. Through website visualization, we compared how the models performed in different situations, such as when a new user had only one interaction and when the user interacted with more products over time.

One limitation of the current user behavior-based recommendation system is handling users with multiple complicated preferences. In real e-commerce websites, a user may not only be interested in one type of product. For example,

the same user may interact with Apple products, watches, clothes, and cosmetics at different times.

This can make recommendations more difficult because the model may not clearly know which interests should be prioritized. If the user recently interacted with many Apple products, the model may focus too much on Apple-related recommendations and ignore the user's other interests. On the other hand, if the user interacts with many different categories, the recommendations may become mixed and less focused.

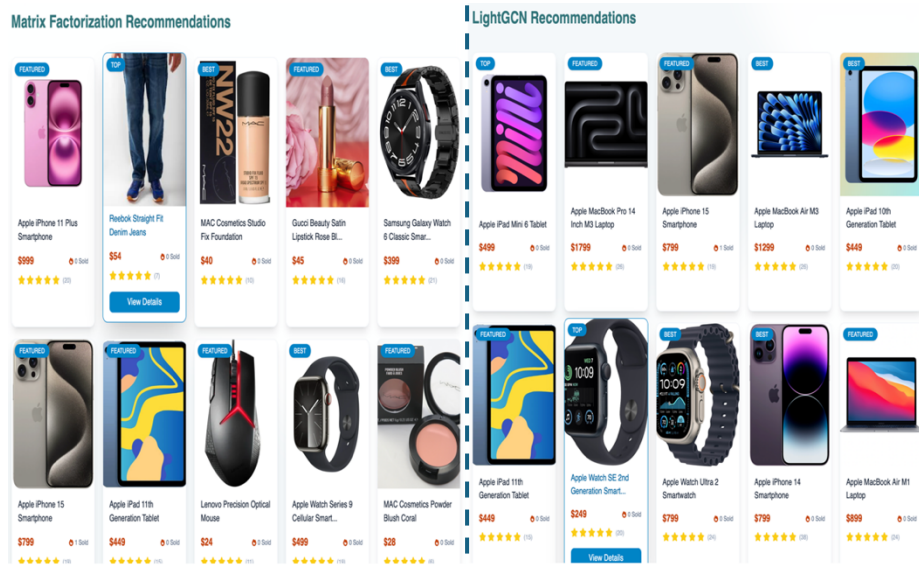


Figure 24 : An Apple-preferred user shifted attention to Cosmetic

These show a weakness of user behavior-based recommendations when a user changes their interests over time. In this case, the user previously interacted with Apple products (Figure 17) but then shifted attention to cosmetics and interacted with three cosmetic items.

From the LightGCN result, most recommended products are still Apple-related, such as iPhone, iPad, MacBook, and Apple Watch. This shows that LightGCN is still strongly influenced by the user's earlier Apple interactions and may not adapt to the new cosmetic interest of the user.

From the Matrix Factorization result, the model starts to include some cosmetic products, such as foundation and blush, which means it can reflect the new interest to some extent. However, the recommendations are still mixed with less

related items, such as jeans, a mouse, and a Samsung watch. This shows that the model also has difficulty giving a clear and focused result.

A possible solution is to use a multi-interest model such as ComiRec⁴. Instead of representing a user with only one preference, ComiRec can learn multiple interest vectors for the same user. This could help the system recommend more balanced and personalized products when user preferences are complicated or change over time.

⁴ ComiRec (Controllable Multi-Interest Framework for Recommendation) is an advanced Deep Learning framework used in large-scale e-commerce and content recommender systems. It solves the problem of predicting what a user wants next by capturing their multiple, distinct interests.

5. References

- [1] Y. Gu, Y. Sun, N. Jiang, B. Wang, and T. Chen, “Learning Global Term Weights for Content-based Recommender Systems,” Proceedings of the 25th International Conference on World Wide Web, 2016.
- [2] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” Computer, vol. 42, no. 8, pp. 30–37, 2009.
- [3] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, “LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation,” Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 639–648, 2020.

6. Appendix

The complete project source code and additional implementation details are available in the GitHub repository: [here](#)